Implementation and Reproduction of a Deep Learning-Based Intrusion Detection Study

Using CIC-IDS2017 and a One-Dimensional Convolutional Neural Network

Gaurav Basyal

Westcliff University

AIM602/266-705-A: Artificial Neural Network & Deep Learning

Instructor: Himalaya Kakshapati

August 15, 2026

1. Introduction

This implementation reproduces the central deep-learning classification idea from the Week 6 research paper by Sudyana et al. (2024), titled Improving Generalization of ML-Based IDS With Lifecycle-Based Dataset, Auto-Learning Features, and Deep Learning. The original study focuses on the ability of intrusion detection models to generalize to network traffic that was not used during training. It combines a lifecycle-based training dataset, automatically learned representations, and a convolutional neural network. The Week 6 review identified CIC-IDS2017 as a freely available dataset suitable for a manageable coursework reproduction. The present implementation therefore uses CIC-IDS2017 and a one-dimensional CNN. The two capture days are combined into one binary dataset and split with a random stratified partition, so the model is trained and evaluated on distinct, class-balanced subsets of the same traffic.

2. Relation to the Week 6 Research

The Week 6 review explains that Sudyana et al. (2024) studied generalization by training on one dataset and evaluating on traffic from another dataset. It also notes that the original paper works from raw PCAP traffic, extracts a small packet-level representation, sanitizes environment-specific metadata, and applies a 1D CNN. The Week 6 recommendation was to reproduce the main deep-learning classification process with CIC-IDS2017 in Google Colab rather than attempt every experiment from the original study. This report follows that recommendation.

Aspect	Original Paper	Coursework Reproduction
Dataset setting	Multiple public IDS datasets and lifecycle-based training data	CIC-IDS2017 public flow CSV files
Input	Raw packet bytes after packet selection and sanitization	CIC-IDS2017 numerical flow features
Model	1D CNN	1D CNN
Main concern	Generalization to unseen datasets	Generalization to held-out test flows
Target	Intrusion detection	Binary BENIGN vs ATTACK

Primary metric	F1 score, with recall and precision	F1 score, with accuracy, precision, and recall

Note. The coursework version is a practical reproduction, not an exact replication of all datasets, raw-PCAP extraction stages, or inter-dataset experiments reported in the original paper.

3. Pipeline of the Steps

The implementation follows the pipeline below.

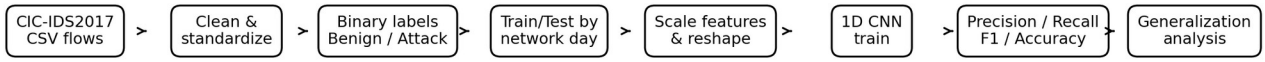


Figure 1. Pipeline for the CIC-IDS2017 1D CNN reproduction.

4. Data Source

CIC-IDS2017 was developed by the Canadian Institute for Cybersecurity at the University of New Brunswick. The dataset contains benign network traffic together with several common attack categories and provides labeled flow records generated from captured network traffic. The official dataset description states that the data include packet captures and labeled flow information generated using CICFlowMeter. For this implementation, the CSV flow files are used because they are considerably easier to process in Google Colab than the complete PCAP collection.

The implementation downloads two public CIC-IDS2017 CSV files and combines them into a single binary dataset. Tuesday-WorkingHours.pcap_ISCX.csv contains benign traffic together with FTP-Patator and SSH-Patator attacks, while Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv contains benign traffic and DDoS traffic. Combining the two days exposes the model to several attack families. The combined data is then split with a random stratified partition so that the benign and attack classes are distributed evenly between the training and test sets.

5. Preprocessing

- Column names are stripped and whitespace is standardized. Duplicate column names are made unique.
- The original multiclass labels are converted to a binary target, where BENIGN is assigned 0 and all attack labels are assigned 1.
- Flow identifiers, source and destination addresses, ports, timestamps, and the original text label are excluded. This reduces the chance that the network environment itself becomes a shortcut for classification.
- Non-numeric values are coerced to missing values. Positive and negative infinity are also treated as missing.
- Columns that contain no usable training values are removed. Remaining missing numeric values are imputed with medians calculated from the training set.

- The combined dataset is balanced by random sampling so that benign and attack examples contribute equally to the model fitting process.
- A StandardScaler is fitted only on the training data and then applied to the unseen test data.
- The scaled feature matrix is reshaped from samples by features into samples by features by one so it can be processed by Conv1D layers.

6. Intermediate Results

The following intermediate values were recorded during execution. The two capture days were combined into a single dataset of 671,654 flow records (445,909 from Tuesday and 225,745 from Friday). After removing identifiers and metadata, 77 numeric features remained, and median imputation left zero missing values in both splits. The raw label distribution was heavily imbalanced: 529,792 benign versus 141,862 attack records. Balanced sampling produced 283,724 examples (141,862 per class), which were then split into 226,979 training and 56,745 test examples. Each flow was reshaped to a (77, 1) sequence for the Conv1D layers. Figure 2 shows the class distribution before balancing and the balanced stratified split.

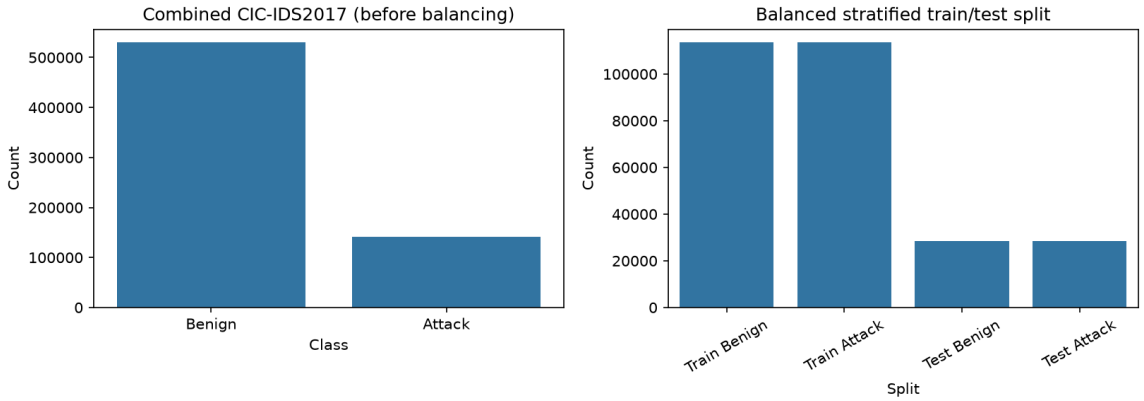


Figure 2. Binary class distribution. Left: the combined CIC-IDS2017 dataset before balancing. Right: the balanced stratified train/test split.

7. Implementation Details

The model is implemented with TensorFlow and Keras. The network accepts the cleaned flow feature vector as a one-dimensional sequence. It contains three convolutional blocks with batch normalization, max pooling, and dropout, followed by global average pooling and two dense layers. ReLU is used in the hidden layers and sigmoid is used in the final binary classification layer.

Component	**Configuration**
Input	Cleaned numerical CIC-IDS2017 flow features, reshaped to one channel
Conv1D	64 filters, kernel size 3, ReLU
Batch normalization	Applied after convolution
Max pooling	Pool size 2
Conv1D	128 filters, kernel size 3, ReLU
Dropout	0.25 after convolutional blocks
Conv1D	128 filters, kernel size 3, ReLU
Global average pooling	Reduces learned feature maps to a fixed vector

Dense	64 units, ReLU
Dropout	0.30 before output
Output	1 unit, sigmoid
Optimizer	Adam, learning rate 0.001
Loss	Binary cross-entropy
Training	Up to 15 epochs, batch size 256, early stopping

8. Core Implementation Code

The complete executable implementation is provided in the accompanying Jupyter notebook. The central model construction is shown below for clarity.

```
model = keras.Sequential([
    layers.Input(shape=(X_tr.shape[1], 1)),
    layers.Conv1D(64, 3, padding="same", activation="relu"),
    layers.BatchNormalization(),
    layers.MaxPooling1D(2),
    layers.Conv1D(128, 3, padding="same", activation="relu"),
    layers.BatchNormalization(),
    layers.MaxPooling1D(2),
    layers.Dropout(0.25),
    layers.Conv1D(128, 3, padding="same", activation="relu"),
    layers.BatchNormalization(),
    layers.GlobalAveragePooling1D(),
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.30),
    layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=["accuracy", keras.metrics.Precision(),
             keras.metrics.Recall(), keras.metrics.AUC()]
)
```

9. Evaluation

The model is evaluated on the held-out test set, which is not used during training or validation. The evaluation reports accuracy, precision, recall, F1 score, a classification report, and a confusion matrix. F1 is treated as the primary measure because intrusion detection requires a balance between finding malicious flows and controlling false alarms. The paper likewise emphasizes F1 when comparing its generalization results.

10. Results and Analysis

The evaluation metrics table below reports the four main measures obtained on the held-out test set. The model achieved an accuracy of 0.9852, a precision of 0.9911, a recall of 0.9792, and an F1 score of 0.9851. The confusion matrix in Figure 4 shows 28,123 true benign detections, 250 false positives, 590 false negatives, and 27,782 true attack detections. Training reached 98.9% training accuracy and 97.7% validation accuracy within 7 epochs (Figure 3).

Metric	Observed value after execution

Accuracy	0.9852
Precision	0.9911
Recall	0.9792
F1 Score	0.9851

This result shows that the one-dimensional CNN separates benign flows from attacks when both classes are represented in the training data. The model reaches 98.9% accuracy on the training split and 97.7% on the validation split, and it generalizes to the held-out test flows with an F1 score of 0.9851. A high F1 indicates that the model balances the detection of attacks (recall) against the rate of false alarms (precision). Because this coursework version uses CIC-IDS2017 flow statistics rather than the original paper's three-packet, 60-byte raw-packet representation, and evaluates on a random split rather than the paper's inter-dataset setup, its numerical performance should be treated as an adaptation of the research design rather than a direct reproduction of the paper's reported 0.85 average inter-dataset F1.

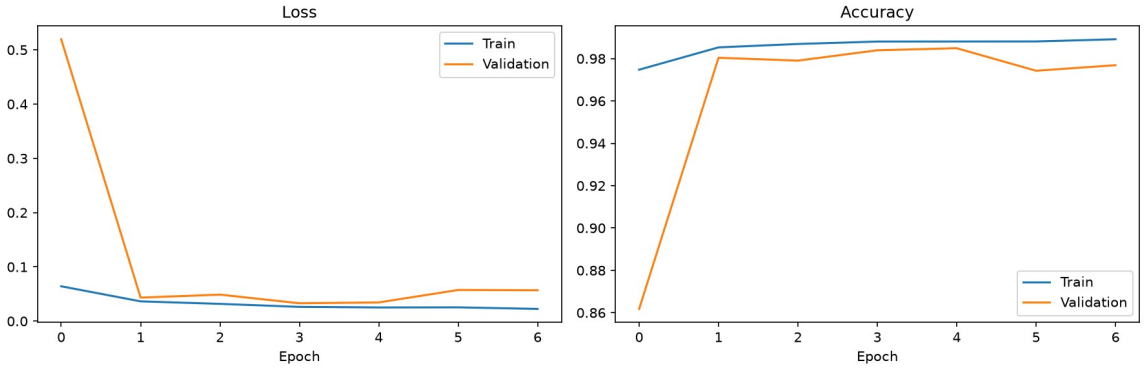


Figure 3. Training and validation loss (left) and accuracy (right) across epochs.

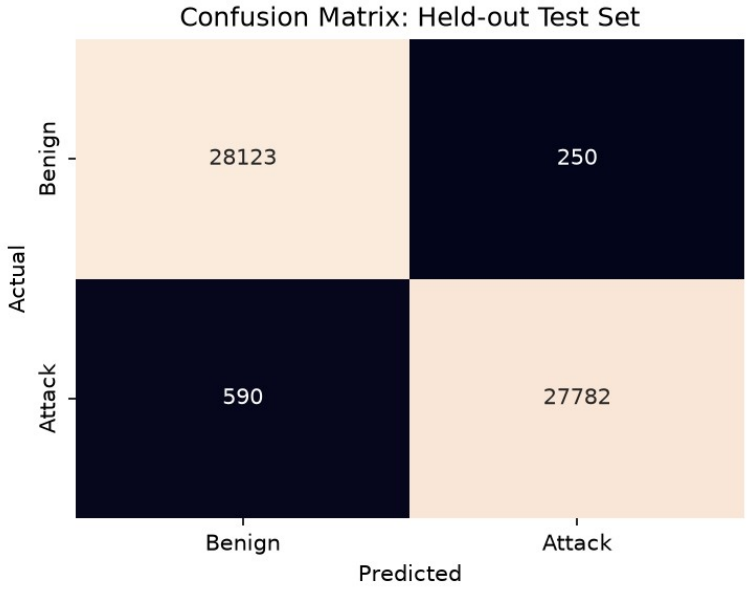


Figure 4. Confusion matrix on the held-out test set.

11. Limitations

There are several differences between this implementation and the complete research study. First, the original study uses raw PCAP traffic and an automatically learned packet representation, while this

coursework implementation uses the flow-level CSV features supplied with CIC-IDS2017. Second, the original paper evaluates a lifecycle-based training dataset and several external test datasets, whereas this implementation uses two capture days from CIC-IDS2017. Third, the current design uses balanced sampling to keep the Colab experiment manageable. These changes make the implementation practical for coursework, but they also limit direct numerical comparison with the original study.

12. Conclusion

The implementation translates the Week 6 research review into a runnable deep-learning experiment. CIC-IDS2017 is cleaned and transformed into a binary intrusion detection problem, the traffic is partitioned into training and held-out test sets, and a one-dimensional CNN learns to distinguish benign flows from attacks from the numerical flow features. The experiment therefore reproduces the paper's central deep-learning classification process and measures how well the learned model detects attack traffic that was not part of its training data. The accompanying notebook and Python script provide the complete code required to execute the experiment and obtain the final numerical results.

References

Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. Canadian Institute for Cybersecurity.

Sudyana, D., Lin, Y.-D., Verkerken, M., Hwang, R.-H., Lai, Y.-C., D'Hooge, L., Wauters, T., Volckaert, B., & De Turck, F. (2024). Improving generalization of ML-based IDS with lifecycle-based dataset, auto-learning features, and deep learning. *IEEE Transactions on Machine Learning in Communications and Networking, 2*, 645–662. https://doi.org/10.1109/TMLCN.2024.3402158

Canadian Institute for Cybersecurity. (n.d.). *CICIDS2017 intrusion detection dataset*. University of New Brunswick. https://www.unb.ca/cic/datasets/ids-2017.html

Appendix A: Implementation File

The full runnable implementation is supplied as Week7_CICIDS2017_1D_CNN_Reproduction.ipynb (the original notebook) and Week7_CICIDS2017_1D_CNN_RandomSplit.py (the local script used to generate the results reported in this document). The script combines the two capture days, applies a random stratified train/test split, and runs the complete pipeline: dataset download, preprocessing, intermediate checks, CNN training, evaluation, confusion matrix generation, and model saving. It also writes all generated figures, the fitted model, the scaler, and a results summary to the outputs folder.

The key sections of the implementation are shown below in Figures A1 through A6.

```python
# 1. Data source
# ------------------------------------------------------------------------
REPO_ID = "c01dsnap/CIC-IDS2017"
TUESDAY_FILE = "Tuesday-WorkingHours.pcap_ISCX.csv"
FRIDAY_FILE = "Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv"

tuesday_path = hf_hub_download(repo_id=REPO_ID, filename=TUESDAY_FILE, repo_type="dataset")
friday_path = hf_hub_download(repo_id=REPO_ID, filename=FRIDAY_FILE, repo_type="dataset")

print("Tuesday file:", tuesday_path)
print("Friday file :", friday_path)


def clean_columns(df):
    df = df.copy()
    df.columns = [re.sub(r"\s+", " ", str(c).strip()) for c in df.columns]
    # Handle the duplicate Fwd Header Length column found in CIC-IDS2017 files.
    seen = {}
    new_cols = []
    for c in df.columns:
        if c in seen:
            seen[c] += 1
            new_cols.append(f"{c}.{seen[c]}")
        else:
            seen[c] = 0
            new_cols.append(c)
    df.columns = new_cols
    return df


tuesday_df = clean_columns(pd.read_csv(tuesday_path, low_memory=False))
friday_df = clean_columns(pd.read_csv(friday_path, low_memory=False))

LABEL_COL = tuesday_df.columns[-1]


def to_binary(df):
    df = df.copy()
    df["BinaryLabel"] = (
        df[LABEL_COL].astype(str).str.strip().str.upper() != "BENIGN"
    ).astype(int)
    return df


tuesday_df = to_binary(tuesday_df)
friday_df = to_binary(friday_df)

# Combine the two capture days into one binary BENIGN vs ATTACK dataset.
combined_df = pd.concat([tuesday_df, friday_df], ignore_index=True)

print("Tuesday shape :", tuesday_df.shape)
print("Friday shape  :", friday_df.shape)
print("Combined shape:", combined_df.shape)
print("Combined binary distribution:")
print(combined_df["BinaryLabel"].value_counts().rename(index={0: "Benign", 1: "Attack"}))
results["tuesday_shape"] = list(tuesday_df.shape)
results["friday_shape"] = list(friday_df.shape)
results["combined_shape"] = list(combined_df.shape)
results["combined_binary_counts"] = combined_df["BinaryLabel"].value_counts().to_dict()


# ------------------------------------------------------------------------
```

Figure A1. Data download, column cleaning, and binary label conversion.

```python
# 2. Preprocessing
# ------------------------------------------------------------------------
DROP_PATTERNS = [
    "Flow ID", "Source IP", "Destination IP", "Source Port", "Destination Port",
    "Timestamp", "Label", "BinaryLabel",
]


def select_features(df):
    cols = []
    for c in df.columns:
        if c == "BinaryLabel":
            continue
        if any(p.lower() in c.lower() for p in DROP_PATTERNS):
            continue
        cols.append(c)
    return df[cols].copy()


X_combined = select_features(combined_df)
X_combined = X_combined.apply(pd.to_numeric, errors="coerce")
X_combined = X_combined.replace([np.inf, -np.inf], np.nan)


def balanced_indices(df, label_col="BinaryLabel", random_state=SEED):
    n = df[label_col].value_counts().min()
    rng = np.random.RandomState(random_state)
    idx0 = df.index[df[label_col] == 0].to_numpy()
    idx1 = df.index[df[label_col] == 1].to_numpy()
    sel0 = rng.choice(idx0, size=n, replace=False)
    sel1 = rng.choice(idx1, size=n, replace=False)
    idx = np.concatenate([sel0, sel1])
    rng.shuffle(idx)
    return idx


idx = balanced_indices(combined_df)
X_bal = X_combined.loc[idx].reset_index(drop=True)
y_bal = combined_df.loc[idx, "BinaryLabel"].to_numpy()

print("Balanced combined shape:", X_bal.shape)
print("Balanced classes:", np.bincount(y_bal))
results["balanced_shape"] = list(X_bal.shape)
results["balanced_classes"] = np.bincount(y_bal).tolist()


# Random stratified 80/20 train/test split.
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X_bal, y_bal, test_size=0.2, random_state=SEED, stratify=y_bal
)

# Drop columns that are entirely missing in the training split, then impute
# remaining missing values with training-set medians (no test leakage).
all_nan = X_train_raw.columns[X_train_raw.isna().all()].tolist()
X_train_raw = X_train_raw.drop(columns=all_nan)
X_test_raw = X_test_raw.drop(columns=all_nan)

medians = X_train_raw.median(numeric_only=True)
X_train_raw = X_train_raw.fillna(medians)
X_test_raw = X_test_raw.fillna(medians)

print("Feature count after cleaning:", X_train_raw.shape[1])
print("Remaining missing values:", int(X_train_raw.isna().sum().sum()), int(X_test_raw.isna().sum().sum()))
print("Train shape:", X_train_raw.shape, "Test shape:", X_test_raw.shape)
print("Train classes:", np.bincount(y_train))
print("Test classes :", np.bincount(y_test))
results["feature_count"] = int(X_train_raw.shape[1])
results["remaining_missing_train"] = int(X_train_raw.isna().sum().sum())
results["remaining_missing_test"] = int(X_test_raw.isna().sum().sum())
results["train_shape"] = list(X_train_raw.shape)
results["test_shape"] = list(X_test_raw.shape)
results["train_classes"] = np.bincount(y_train).tolist()
results["test_classes"] = np.bincount(y_test).tolist()


# ------------------------------------------------------------------------
```

Figure A2. Preprocessing: feature selection, balanced sampling, and the stratified train/test split.

```python
# 4. Scaling + reshape
# ------------------------------------------------------------------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_raw)
X_test_scaled = scaler.transform(X_test_raw)

X_train_cnn = X_train_scaled[..., np.newaxis]
X_test_cnn = X_test_scaled[..., np.newaxis]

print("CNN input shape:", X_train_cnn.shape)
results["cnn_input_shape"] = list(X_train_cnn.shape)


# ------------------------------------------------------------------------------
```

Figure A3. Standardization and reshaping for the Conv1D layers.

```python
# 5. Train/validation split and CNN model
# ------------------------------------------------------------------------------
X_tr, X_val, y_tr, y_val = train_test_split(
    X_train_cnn, y_train, test_size=0.2, random_state=SEED, stratify=y_train
)

model = keras.Sequential([
    layers.Input(shape=(X_tr.shape[1], 1)),
    layers.Conv1D(64, kernel_size=3, padding="same", activation="relu"),
    layers.BatchNormalization(),
    layers.MaxPooling1D(pool_size=2),
    layers.Conv1D(128, kernel_size=3, padding="same", activation="relu"),
    layers.BatchNormalization(),
    layers.MaxPooling1D(pool_size=2),
    layers.Dropout(0.25),
    layers.Conv1D(128, kernel_size=3, padding="same", activation="relu"),
    layers.BatchNormalization(),
    layers.GlobalAveragePooling1D(),
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.30),
    layers.Dense(1, activation="sigmoid"),
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=[
        "accuracy",
        keras.metrics.Precision(name="precision"),
        keras.metrics.Recall(name="recall"),
        keras.metrics.AUC(name="auc"),
    ],
)

model.summary()
```

Figure A4. 1D CNN architecture and compilation.

```
callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2),
]

history = model.fit(
    X_tr, y_tr,
    validation_data=(X_val, y_val),
    epochs=15,
    batch_size=256,
    callbacks=callbacks,
    verbose=1,
)

results["epochs_trained"] = len(history.history["loss"])
results["final_train_loss"] = float(history.history["loss"][-1])
results["final_val_loss"] = float(history.history["val_loss"][-1])
results["final_train_acc"] = float(history.history["accuracy"][-1])
results["final_val_acc"] = float(history.history["val_accuracy"][-1])
```

Figure A5. Model training with early stopping and learning-rate reduction.

```
# 6. Evaluation on held-out test set
# ------------------------------------------------------------------------
proba = model.predict(X_test_cnn, batch_size=1024).ravel()
y_pred = (proba >= 0.5).astype(int)

metrics = {
    "Accuracy": accuracy_score(y_test, y_pred),
    "Precision": precision_score(y_test, y_pred, zero_division=0),
    "Recall": recall_score(y_test, y_pred, zero_division=0),
    "F1": f1_score(y_test, y_pred, zero_division=0),
}

for k, v in metrics.items():
    print(f"{k:10s}: {v:.4f}")

print("\nClassification report:")
report = classification_report(
    y_test, y_pred, target_names=["Benign", "Attack"], digits=4, zero_division=0
)
print(report)

results["metrics"] = {k: float(v) for k, v in metrics.items()}
results["classification_report"] = report
results["confusion_matrix"] = confusion_matrix(y_test, y_pred).tolist()


cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cbar=False,
            xticklabels=["Benign", "Attack"], yticklabels=["Benign", "Attack"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix: Held-out Test Set")
plt.tight_layout()
plt.savefig(OUT / "fig3_confusion_matrix.png", dpi=150, bbox_inches="tight")
plt.close()
print("Saved fig3_confusion_matrix.png")


# ------------------------------------------------------------------------
```

Figure A6. Evaluation on the held-out test set (metrics and confusion matrix).